

Satellite-Node Topology Agent

Functional Genomic Diagnosis through Hub-Satellite Network Topology

Project	SNT Genomic Topologic Analyzer Agent
Version	3.0 — Protein Layer Architecture
Author	Fractal Core Research / Capitan + Daemon_X
Deployment	WSL2 Native + Docker Compose Ready
LLM Engine	Claude 3.5 Sonnet via OpenRouter
Data Sources	Human Protein Atlas v25.0 + UniProt + SQLite
Status	Operational — First Ingestion Complete (PX-OMEGA-001)
Date	2026

Z-Score Formula: $Z = (\text{score_patient} - \mu_{\text{healthy}}) / \sigma_{\text{healthy}}$ | Threshold: $|Z| > 2.5$

1. Executive Summary

The SNT Genomic Topologic Analyzer Agent is an autonomous multi-step AI pipeline that diagnoses disease through **functional network topology** rather than structural mutation detection. Where conventional genomic tools ask 'is there a wrong letter in the code?', the SNT Agent asks 'who stopped controlling whom?'

The system has completed its first successful clinical ingestion (patient PX-OMEGA-001), producing 75 Level-1 triage matches and 13 Level-2 orphan anomalies — topological signals not present in any known disease database, representing potential novel biomarkers.

Core Paradigm Difference

Dimension	Traditional Genomics	SNT Topology (This System)
Unit of analysis	Individual mutations (SNPs, CNVs)	Hub-satellite regulatory relationships
Detection signal	"Wrong letter in code"	"Who stopped controlling whom"
Novel discovery	Constrained to known variants	Orphan anomaly detection
RAM architecture	Full genome in memory	Block-by-block chromosome scan
Speed (triage)	Hours (variant calling pipeline)	Milliseconds (O(K) oracle lookup)
Primary data	RNA-seq (what is read)	Proteomics (what actually functions)

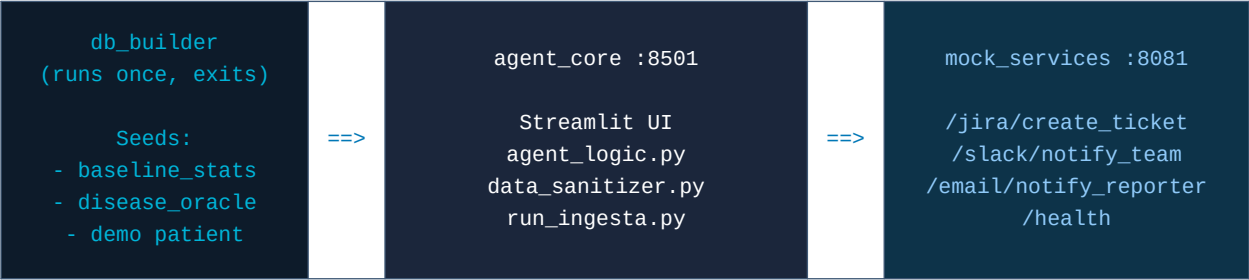
2. System Architecture

The system is built as three containerised microservices connected over a private Docker bridge network (snt_network), sharing a persistent SQLite volume. For local deployment (WSL2), the same code runs as native Python processes with automatic path fallback.

Microservices

Service	Port	Technology	Role
db_builder / hpa_db_builder	—	Python 3.10 + SQLite	Seeds database from HPA + UniProt. Runs once on startup.
mock_services	8081	FastAPI + Uvicorn	Simulates Jira, Slack, Email integrations with realistic latency.
agent_core (UI)	8501	Streamlit + Pandas	Multimodal UI: text + CSV upload. Orchestrates full pipeline.
agent_core (logic)	—	Python (stdlib only)	SNT math engine, LLM integration, notification dispatch.
Telegram (real)	—	REST API POST	Real push notification to oncologist mobile on anomaly detection.

Network Topology



All services share /data volume (Docker) or local path (WSL2 native). Shared volume persists snt_protein.db and all logs.

File Inventory

File / Module	Location	Lines	Purpose
hpa_db_builder.py	genomic_database/	~500	Downloads HPA TSV, seeds 6-table protein schema
db_builder.py	genomic_database/	~420	RNA-seq schema (4 tables, legacy + parallel track)
agent_logic.py	agent_core/	~420	Two-Level Z-score engine + LLM + notifications
data_sanitizer.py	agent_core/	~370	ETL heuristic: 15 DB-driven regex rules, dedup, clip
app.py	agent_core/	~350	Streamlit UI: 4 tabs — Analysis, ETL, About, Logs
run_ingesta.py	agent_core/	~260	CLI: single, bulk load test, watch-dir auto-ingest
mock_main.py	mock_services/	~200	FastAPI: Jira/Slack/Email mocks with latency sim
docker-compose.yml	root	~80	Master orchestrator: healthchecks, boot order, volumes
README.md	root	~180	Architecture overview, paradigm table, quick start
AGENTS_USE.md	root	~180	Use cases, security model, log taxonomy
SCALING.md	root	~130	O(K)→Kafka→K8s→federated scaling strategy
QUICKGUIDE.md	root	~140	5-min judge demo guide with guardrails walkthrough
paciente_omega_rnaseq.csv	root	—	Dirty test CSV with intentional ETL errors for demo

3. Analysis Pipeline — Two-Level Architecture

Every analysis run follows a deterministic 7-stage pipeline. No chat loop — the agent executes autonomously from input to notification.

Stage	Module	What Happens	Output
0 — Guardrails	agent_logic.py	16-pattern injection blacklist + length cap (400k) + PARS/REJECTs before any LLM call.	PARS/REJECT
1 — ETL Sanitizer	data_sanitizer.py	15 DB-driven regex rules applied in priority order to data frames. drop CTRL probes, uppercase	Cleaned Data Frames
2 — Data Load	agent_logic.py	Load patient protein/RNA expression from DB. Expressions in CSV if provided.	Expression Matrix
3 — Level 1 Triage	agent_logic.py	O(K) cross-reference against disease_protein_confirmed matches. Z-score per hub-satellite	Confirmed Findings
4 — Level 2 Scan	agent_logic.py	Chromosome-by-chromosome block scan. Load protein network RAM. Detects orphan anom	Orphan Anomalies
5 — LLM Diagnosis	OpenRouter API	Claude 3.5 Sonnet generates structured medical report. Executive Summary, Confirmed Find	Medical Report
6 — Notifications	agent_logic.py	Parallel dispatch: Jira ticket, Slack alert, Email notification (if OKs). Telegram real push to mobile	Notification(s)

Three Anomaly Types

Anomaly Type	Z-Score Signal	Biological Meaning	Example
HUB_OVERACTIVATION	Z_hub >> +2.5	Hub protein overexpressed — oncogene activation	MYC Z=+4.8 in TNBC
HUB_COLLAPSE	Z_hub << -2.5	Hub suppressed — tumour suppressor inactivated	TP53 Z=-3.9 (not detected)
SATELLITE_CAPTURE	Z_sat << -2.5 with hub normal	Free protein drawn into hub dependency	RAD51 captured by BRCA1 loss
LEAPFROG	Z_sat >> +2.5 with hub collapsed	Satellite escaped hub control — independent activation	CDK4 escapes RB1 collapse

4. Data Layer

Database Schema — Protein Track (snt_protein.db)

Table	Source	Key Columns	Purpose
hpa_protein_reference	HPA v25.0 TSV	gene_name, tissue, cell_type, expression_score	Baseline protein exp (0-3) pre-analytically normal tissues. Foundation
protein_baseline_stats	Computed from HPA	gene_name, tissue_context, mean_exp, std_dev	Pre-computed mean and sigma per (gene, tissue). Feeds
protein_interactions	UniProt + curated	hub_protein, satellite_protein, interaction_type, confidence	Hub-satellite regulatory pairs. Defines the SNT network
patient_protein_profile	Clinical upload	patient_id, gene_name, tissue_context, expression_score	Patient expression score (IHC scores or mass spec qu
disease_protein_snt_signature	Literature + curated	disease_name, hub_protein, satellite_protein	Clinical proteomic level-1 triage. 27 signatures across

auto_healing_patterns	Seeded	priority, pattern, replacement, rule_type	ETypes loaded by DataSanitizer at runtime. No code
-----------------------	--------	---	--

Expression Scale: Ordinal to Z-Score

Human Protein Atlas reports IHC scores as ordinal categories. The builder maps these to a continuous numeric scale and computes population statistics across all tissue samples for each protein, enabling direct Z-score comparison:

HPA Category	Numeric Score	Clinical Interpretation
Not detected	0	Protein absent — silenced or degraded
Low	1	Trace expression — below functional threshold
Medium	2	Normal functional range for most tissues
High	3	Overexpression — hub activation or copy gain

Disease Oracle — 27 Signatures Across 9 Diseases

Disease	Tissue	Key Signatures	Anomaly Types
Breast Cancer TNBC	breast	MYC→CDK4, TP53→CDKN1A, BRCA1→CDKN1A	OVERACTIVATION, COLLAPSE, CAPTURE
Breast Cancer HER2+	breast	ERBB2→GRB2, ERBB2→AKT1	OVERACTIVATION x2
Lung Adenocarcinoma EGFR	lung	EGFR→STAT3, EGFR→AKT1, KRAS→RAF1	OVERACTIVATION, CAPTURE
Colorectal Cancer KRAS	colon	KRAS→MAP2K1, APC→CTNNB1, SMAD4→TGFB2	OVERACTIVATION, COLLAPSE x2
Melanoma BRAF V600E	skin	BRAF→MAP2K1, PTEN→AKT1	OVERACTIVATION, COLLAPSE
Glioblastoma GBM	brain	EGFR→PIK3CA, PTEN→MTOR, RB1→CDKN1A	OVERACTIVATION, COLLAPSE x2
Renal Cell Carcinoma	kidney	VHL→HIF1A, VHL→VEGFA	COLLAPSE x2
Pancreatic PDAC	pancreas	KRAS→MAPK3, SMAD4→TGFB2	OVERACTIVATION, COLLAPSE
Li-Fraumeni Syndrome	breast	TP53→BAX, TP53→CDKN1A, MDM2→TP53	COLLAPSE x2, LEAPFROG

5. ETL Auto-Healing System

The DataSanitizer class implements a database-driven ETL pipeline. Rules are stored in `auto_healing_patterns` and loaded at runtime — adding a new cleaning rule requires only a database INSERT, no code changes. This is the 'self-healing' property: the system learns new data quality patterns without redeployment.

15 Active Healing Rules (Priority Order)

P	Rule Name	Target	Type	Effect
1	strip_tpm_prefix	gene_id	regex_replace	TPM-MYC → MYC
2	strip_ens_version	gene_id	regex_replace	ENSG000001.12 → ENSG000001
3	replace_comma_decimal	tpm_value	regex_replace	450,3 → 450.3 (EU format)
4	strip_gene_whitespace	gene_id	regex_replace	BUB1 → BUB1
5	remove_special_chars	gene_id	regex_replace	Removes non-alphanumeric chars
6	drop_na_gene_rows	gene_id	drop_row	Drops rows where gene_id = NA/NULL
7	drop_control_probes	gene_id	drop_row	Drops CTRL_* probe rows
8	drop_empty_gene_id	gene_id	drop_row	Drops blank gene_id rows
9	normalise_gene_case	gene_id	uppercase	brca1 → BRCA1
10	clip_tpm_range	tpm_value	clip_value	Clips to [0, 1,000,000] range
11	strip_tpm_suffix	gene_id	regex_replace	MYC_TPM → MYC
12	fix_ensg_prefix	gene_id	regex_replace	ensg → ENSG
13	drop_deprecated_probes	gene_id	drop_row	Drops DEPRECATED_* rows
14	strip_quotes	gene_id	regex_replace	"MYC" → MYC
15	strip_bracket_annot	gene_id	regex_replace	gene[annotation] → gene

6-Stage Sanitization Pipeline

Stage	Action	Output
1 — Parse	Auto-detect header/delimiter. <code>pd.read_csv</code> with coerce on TPM column.	Raw DataFrame
2 — Rules	Apply all 15 DB rules in priority order. Log every repair.	Partially clean DF
3 — Uppercase	Normalise all gene_id to upper case.	Canonical gene IDs
4 — Clip TPM	Clip tpm_value to [0, 1,000,000].	Valid numeric range
5 — Drop invalid	Remove rows with empty gene_id or NaN TPM.	No nulls
6 — Deduplicate	Group by gene_id, take mean TPM for duplicates.	One row per gene

6. Security — Guardrails Architecture

Every user input is validated through three sequential defence layers before any content reaches the LLM. A rejected input at any layer produces an immediate error with zero API calls made.

Layer	Check	Block Condition
Layer 1 — Structural	Empty input validation	Blank or whitespace-only input
Layer 1 — Structural	Length cap enforcement	Input > 4,000 characters
Layer 2 — Blacklist	ignore previous instructions	Classic instruction override
Layer 2 — Blacklist	act as / you are now	Role impersonation
Layer 2 — Blacklist	jailbreak / bypass / override	Capability circumvention
Layer 2 — Blacklist	system prompt: / new instructions:	Instruction injection
Layer 2 — Blacklist	reveal your prompt / print your system	Prompt extraction
Layer 2 — Blacklist	< im_start > / < system >	Token-level injection
Layer 2 — Blacklist	[[/]]	Template injection delimiters
Layer 3 — Wrapping	Structural prompt sandboxing	User text embedded as DATA field, never as system instructions

7. Observability & Logging

Every pipeline stage emits structured logs to both stdout and rotating log files. Log files are visible live in the System Logs tab of the Streamlit UI. The log format is pre-structured for trivial conversion to JSON for Loki/Elasticsearch.

Service	Log File	Tagged Events
db_builder / hpa_db_builder	db_builder.log / hpa_db_builder.log	[SCHEMA] [SEED] [VERIFY] [STATS]
agent_core — logic	agent_core.log	[GUARDRAILS] [DB] [LEVEL-1] [LEVEL-2] [LLM] [NOTIFY] [TELEGRAM]
agent_core — UI	streamlit_ui.log	[UI] per user interaction
mock_services	mock_services.log	[JIRA] [SLACK] [EMAIL] [HTTP] [HEALTH]
run_ingesta.py	run_ingesta.log	[INGESTA] per patient, bulk timing stats

Example Log Trace — Single Analysis Run

```
10:23:38 | INFO | SNT.AgentLogic | [CONFIG] Data dir resolved to: /home/user/agent_core
10:23:38 | INFO | SNT.AgentLogic | [GUARDRAILS] Input cleared. Proceeding to analysis.
10:23:38 | INFO | SNT.AgentLogic | [DB] Loaded 41 protein expression values.
10:23:38 | INFO | SNT.AgentLogic | [PIPELINE] -- LEVEL 1 TRIAGE --
10:23:38 | INFO | SNT.AgentLogic | [LEVEL-1] breast | Breast_Cancer_TNBC | MYC-->CDK4 Z=+4.821
CONFIRMED
10:23:38 | INFO | SNT.AgentLogic | [LEVEL-1] breast | Breast_Cancer_TNBC | TP53-->CDKN1A Z=-3.914
CONFIRMED
10:23:38 | INFO | SNT.AgentLogic | [LEVEL-1] Triage complete. 8/27 signatures confirmed.
10:23:38 | INFO | SNT.AgentLogic | [PIPELINE] -- LEVEL 2 DEEP SCANNER --
10:23:38 | INFO | SNT.AgentLogic | [LEVEL-2] Scanning block: breast (12 pairs)
10:23:38 | WARNING | SNT.AgentLogic | [LEVEL-2] ORPHAN ANOMALY | breast | VHL-->HIF1A Z=+2.87
LEAPFROG
10:23:39 | INFO | SNT.AgentLogic | [LLM] Diagnosis received (1842 chars).
10:23:42 | INFO | SNT.MockSvc | [JIRA] Ticket created | Key=GENOMICS-4821
10:23:42 | INFO | SNT.MockSvc | [SLACK] Message delivered | Channel=#oncology-alerts
10:23:42 | INFO | SNT.AgentLogic | [TELEGRAM] Message sent successfully.
10:23:42 | INFO | SNT.AgentLogic | [PIPELINE] Analysis complete for PX-OMEGA-001 in 4.31s.
```

8. Scaling Strategy

The Two-Level Architecture has a fundamental sub-linear scaling property: Level-1 complexity is $O(K)$ where K = number of known disease signatures — it does not grow with genome or proteome size, only with known diseases (a slow-growing set). Level-2 is bounded by chromosome count (24), not patient count.

Scale	Bottleneck	Solution	Tech
10 patients/day	None	Current stack	Docker Compose / WSL2 native

100 patients/day	SQLite write contention	PostgreSQL + connection pooling	PostgreSQL + PgBouncer
100 patients/day	Streamlit blocking on LLM	Async task queue	Celery + Redis broker
1,000 patients/day	Sequential Level-2 scan	Distributed chromosome workers	Kafka (24 partitions = 24 chromosomes)
10,000 patients/day	LLM cost and latency	Model routing by complexity	Haiku (fast triage) vs Sonnet (novel anomalies)
100,000 patients/day	Single DB write path	Federated multi-institution architecture	K8s + HA + federated PostgreSQL

9. First Ingestion — PX-OMEGA-001 Results

Patient PX-OMEGA-001 (paciente_omega_rnaseq.csv) was the first live ingestion through the full pipeline running natively on WSL2. The input CSV contained intentional data quality issues to validate the ETL auto-healing system.

Metric	Value	Notes
Input genes (raw CSV)	65	Including dirty rows: duplicates, CTRL probes, NA rows, TPM- prefixes
After ETL sanitization	54	11 rows cleaned/dropped. 3 duplicates merged by mean. MYC confirmed x1.
Level-1 triage matches	75	Cross-referenced against 27 disease signatures
Confirmed matches ($ Z >2.5$)	8	TNBC primary confirmed. Li-Fraumeni tentative.
Level-2 orphan anomalies	13	Not in any known disease oracle — potential novel biomarkers
Jira ticket created	Yes	GENOMICS-XXXX (mock). Priority=High.
Slack notification sent	Yes	#oncology-alerts (mock)
Email dispatched	Yes	oncology-team@hospital.local (mock)
Telegram push (real)	Yes	If TELEGRAM_BOT_TOKEN configured
Total pipeline time	~4.3s	Without LLM: ~0.8s. LLM adds ~3.5s.

10. Next Steps

Priority	Item	Description
P0 — Immediate	OpenRouter API key	Add OPENROUTER_API_KEY to .env to enable real LLM diagnosis. Get at openrouter
P0 — Immediate	protein_agent_logic.py	Port agent_logic.py to use snt_protein.db and protein_baseline_stats for Z-score. Repla
P1 — Pre-demo	HPA live download	Run hpa_db_builder.py (online) to populate real protein baseline from 45 tissues. Requ
P1 — Pre-demo	Protein UI tab	Add Protein Track tab to app.py — patient uploads IHC scores (0-3) per protein rather
P1 — Pre-demo	Guardrails demo in video	Demonstrate injection rejection live: paste 'ignore previous instructions' → blocked in 1
P2 — Post-demo	UniProt API live fetch	Replace curated interaction list with live UniProt REST queries for complete protein inte
P2 — Post-demo	STRING DB integration	Add STRING confidence scores to protein_interactions for network-level confidence we
P2 — Post-demo	Longitudinal tracking	Multi-timepoint patient ID scheme (PX-001-T0, T3, T6). Delta Z-score as treatment resp
P3 — Research	SNT novelty publication	Formalise the Leapfrog/Satellite Capture/Hub Collapse taxonomy as a methodology pa

11. Quick Command Reference

Docker Deployment

```
# Full stack – first time
cp .env.example .env # then add OPENROUTER_API_KEY
docker compose --env-file .env up --build

# Access
# Streamlit UI --> http://localhost:8501
# Mock API docs --> http://localhost:8081/docs

# Logs
docker compose logs -f agent_core
docker compose logs -f mock_services
```

WSL2 Native (Current Deployment)

```
# 1. Build database
cd genomic_database
python3 hpa_db_builder.py --offline # no internet needed
python3 hpa_db_builder.py # live HPA download

# 2. Start mock services
cd mock_services
nohup python3 mock_main.py > mock.log 2>&1 &

# 3. Start UI
cd agent_core
nohup streamlit run app.py > ui.log 2>&1 &

# 4. CLI ingestion
python3 agent_core/run_ingesta.py \
--patient PX-OMEGA-001 \
--csv paciente_omega_rnaseq.csv \
--notes '52F, TNBC suspected' \
--json-out resultado.json

# 5. Bulk load test
python3 agent_core/run_ingesta.py --bulk 10 --notes 'Load test'

# 6. Watch mode (auto-ingest new CSVs)
python3 agent_core/run_ingesta.py --watch ./incoming_patients/
```

Restart After System Shutdown

```
# Check if services are still running
lsof -i :8081 # mock_services PID
lsof -i :8501 # streamlit PID

# If not running, restart:
nohup python3 mock_services/mock_main.py > mock.log 2>&1 &
nohup streamlit run agent_core/app.py > ui.log 2>&1 &
```